**BÀI 05.THIẾT KẾ LAYOUT TRONG FLUTTER (tiếp theo)**

## A. MỤC TIÊU:

− Sử dụng được các widget cơ bản của flutter như: Dialog, ListView, GirdView

− Kết hợp được các widget để xây dựng được giao diện hoàn chỉnh

## B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SV:

| STT | Chủng loại – Quy cách vật tư | Số lượng | Đơn vị | Ghi chú |
|---|---|---|---|---|
| 1 | Computer | 1 | 1 | |

## C. NỘI DUNG THỰC HÀNH

### 1. Tóm tắt lý thuyết

#### a. Alert Dialog

**- Đặc điểm:**

✓ Là một widget dùng để hiển thị hộp thoại cảnh báo cho người dùng.

**- Các thuộc tính:**

✓ **title:** Tiêu đề của hộp thoại, thường được hiển thị ở đầu.

✓ **content:** Nội dung chính của hộp thoại, có thể là văn bản, hình ảnh hoặc bất kỳ widget nào khác.

✓ **actions:** Danh sách các nút hành động cho phép người dùng tương tác với hộp thoại, như "OK", "Cancel".

**Ví dụ:**

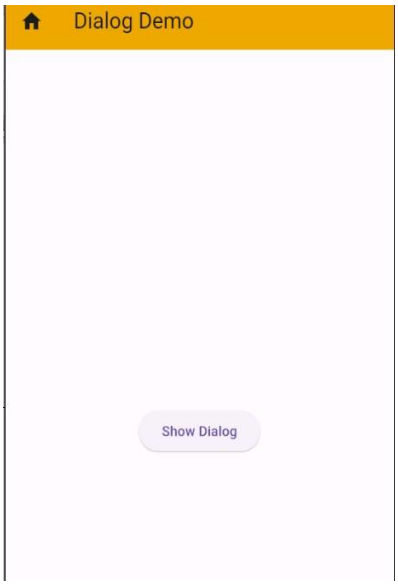
```
class MyDialog extends StatefulWidget {
  const MyDialog({super.key});

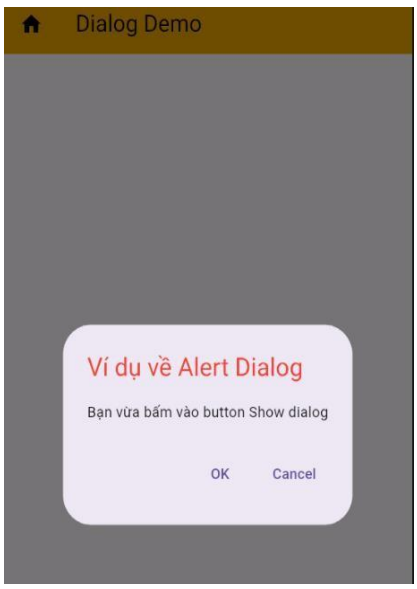
  @override
  State<MyDialog> createState() => _MyDialogState();
}

class _MyDialogState extends State<MyDialog> {
  void _showDialog() {
    showDialog(
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          title: const Text(
            "Ví dụ về Alert Dialog",
            style: TextStyle(color: Colors.red),
          ),
          content: const Text("Bạn vừa bấm vào button Show dialog"),
          actions: <Widget>[
            TextButton(
              onPressed: () {
                Navigator.of(context).pop();
              },
              child: const Text("OK"),
```

```
            ),
            TextButton(
              onPressed: () {
                Navigator.of(context).pop();
              },
              child: const Text("Cancel"),
            ),
          ],
        );
      },
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Dialog Demo"),
        backgroundColor: const Color.fromARGB(255, 239, 168, 4),
        leading: IconButton(
          icon: const Icon(Icons.home),
          onPressed: () {},
        ),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: _showDialog,
          child: const Text("Show Dialog"),
        ),
      ),
    );
  }
}
```



**b. Listview widget**

**- Đặc điểm**

✓ **ListView** là một trong những widget phổ biến nhất trong Flutter, được sử dụng để hiển thị danh sách các mục. Nó có khả năng cuộn và được tối ưu hóa để làm việc với một số lượng lớn các mục. **ListView** có thể hiển thị theo chiều dọc hoặc ngang.

✓ **ListView** có thể tự động scroll khi danh sách các mục vượt quá kích thước màn hình; tối ưu hóa bộ nhớ bằng cách chỉ load các mục khi chúng hiện lên màn hình, giúp giảm thiểu tiêu thụ bộ nhớ; dễ dàng tùy chỉnh: Có thể dễ dàng tùy chỉnh giao diện của các mục trong danh sách.

**- Phân loại Listview**

✓ **ListView:** Đây là ListView cơ bản nhất, có thể chứa một danh sách các widget được tạo trước.

✓ **ListView.builder:** Loại này thường được sử dụng khi số lượng mục không cố định hoặc quá lớn để tải tất cả cùng một lúc. ListView.builder tạo các widget theo yêu cầu trong quá trình cuộn. với 2 tham số:

o **itemCount**: Tổng số item trong list.

o **itemBuilder:** sẽ tạo view cho từng item

✓ **ListView.separated:** thêm chức năng để thêm các widget ngăn cách giữa các mục.

✓ **ListView.custom:** cho phép bạn tùy chỉnh hoàn toàn cách hiển thị các mục trong ListView bằng cách sử dụng SliverChildDelegate

o **SliverChildBuilderDelegate:** Sử dụng khi bạn có một số lượng lớn các mục hoặc khi các mục được tạo ra động (dynamic).

o **SliverChildListDelegate:** Sử dụng khi bạn có một danh sách các mục cố định và không thay đổi

**- Một số thuộc tính**

✓ **children**: Danh sách các widget con sẽ được hiển thị trong ListView.

✓ **itemCount**: Số lượng mục sẽ được hiển thị trong ListView. Thường được sử dụng với ListView.builder.

✓ **itemBuilder**: Hàm xây dựng các mục cho ListView. Thường được sử dụng với ListView.builder.

✓ **scrollDirection**: Xác định hướng cuộn của ListView (ngang hoặc dọc). Mặc định là dọc.

✓ **padding**: Đặt khoảng cách xung quanh các mục trong ListView.

✓ **shrinkWrap**: Xác định xem ListView có nên chỉ chiếm không gian cần thiết để hiển thị nội dung của nó hay không. Mặc định là false.

✓ **physics**: Xác định cơ chế cuộn của ListView

o **NeverScrollableScrollPhysics:** Bỏ chế độ cuộn của ListView

o **BouncingScrollPhysics:** Khi cuộn tới item cuối thì trả về ban đầu

o **ClampingScrollPhysics:** Khi cuộn item cuối có thể tạo ra hiệu ứng

o **FixedExtentScrollPhysics:** Cuộn hình dạng tròn

✓ **controller**: Đối tượng ScrollController để điều khiển cuộn của ListView.

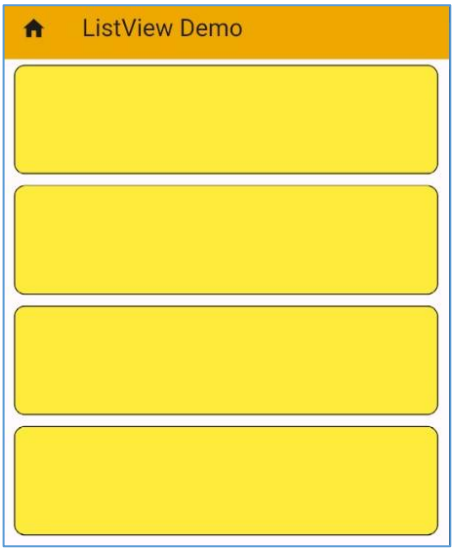
**- Ví dụ:**

```
class MyListViewDeMo extends StatefulWidget {
  const MyListViewDeMo({super.key});

  @override
  State<MyListViewDeMo> createState() => _MyListViewDeMoState();
}

class _MyListViewDeMoState extends State<MyListViewDeMo> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("ListView Demo"),
        backgroundColor: const Color.fromARGB(255, 239, 168, 4),
        leading: IconButton(
          icon: const Icon(Icons.home),
          onPressed: () {},
        ),
      ),
      body: ListView(
        children: <Widget>[
          Container(
            decoration: BoxDecoration(
              border: Border.all(color: Colors.black),
              color: Colors.yellow,
              borderRadius: BorderRadius.circular(10),
            ),
            height: 100,
            margin: const EdgeInsets.only(left: 10, right: 10, bottom: 5, top: 5),
          ),
          Container(
            decoration: BoxDecoration(
              border: Border.all(color: Colors.black),
              color: Colors.yellow,
              borderRadius: BorderRadius.circular(10),
            ),
            height: 100,
            margin: const EdgeInsets.only(left: 10, right: 10, bottom: 5, top: 5),
          ),
          Container(
            decoration: BoxDecoration(
              border: Border.all(color: Colors.black),
              color: Colors.yellow,
              borderRadius: BorderRadius.circular(10),
            ),
            height: 100,
            margin: const EdgeInsets.only(left: 10, right: 10, bottom: 5, top: 5),
          ),
          Container(
```

```
            decoration: BoxDecoration(
              border: Border.all(color: Colors.black),
              color: Colors.yellow,
              borderRadius: BorderRadius.circular(10),
            ),
            height: 100,
            margin: const EdgeInsets.only(left: 10, right: 10, bottom: 5, top: 5),
          ),
        ],
      ),
    );
  }
}
```




Để đơn giản ta có thể tạo 1 lớp ListItem như sau:

```
class MyListItem extends StatelessWidget {
  const MyListItem({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        border: Border.all(color: Colors.black),
        color: Colors.yellow,
        borderRadius: BorderRadius.circular(10),
      ),
      height: 100,
      margin: const EdgeInsets.only(left: 10, right: 10, bottom: 5, top: 5),
    );
  }
}
```

Và sửa lại nội dung thuộc tính **body** như sau:

```
body: ListView(
  children: const <Widget>[
    MyListItem(),
```

```
    MyListItem(),
    MyListItem(),
    MyListItem(),
    MyListItem(),
  ],
)

// Hoặc

body: ListView.builder(
  itemBuilder: (BuildContext context, int index) {
    return const MyListItem();
  },
  itemCount: 5,
)
```

### c. GirdView widget

**- Đặc điểm**

✓ Là một widget cho phép bạn hiển thị các mục dữ liệu theo dạng lưới, tức là các hàng và cột

**- Một số loại GridView thông dụng**

✓ **GridView.count():** Cho phép xác định số lượng cột trong GridView

✓ **crossAxisCount:** Nó được sử dụng để chỉ định số lượng cột trong chế độ xem lưới.

✓ **crossAxisSpacing:** Nó được sử dụng để chỉ định số lượng pixel giữa mỗi widget con được liệt kê trong trục chéo.

✓ **mainAxisSpacing:** Nó được sử dụng để chỉ định số lượng pixel giữa mỗi widget con được liệt kê trong trục chính.

✓ **Padding:** Nó được sử dụng để chỉ định không gian xung quanh toàn bộ danh sách các widget.

✓ **Children:** danh sách các item con của GridView

✓ **GridView.builder():** Cách sử dụng giống với ListView.builder()

✓ **temCount :** xác định lượng dữ liệu sẽ được hiển thị.

✓ **gridDelegate :** xác định khoảng cách giữa các item trong GridView (bắt buộc), có thể nhận các giá trị: SliverGridDelegateWithFixedCrossAxisCount, SliverGridDelegateWithMaxCrossAxisExtent

✓ **itemBuilder :** Hàm xác định cách hiển thị các item.

✓ **children:** Chứa các widget con

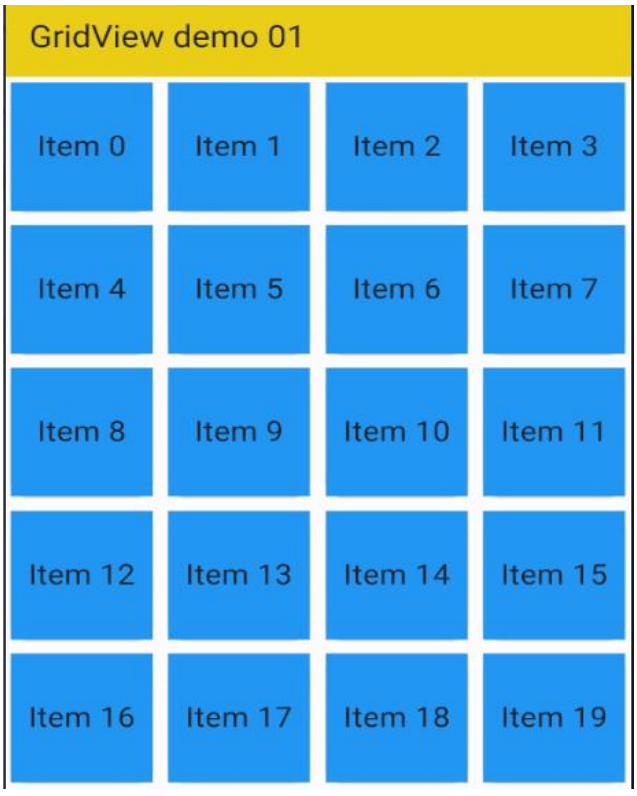
✓ **GridView.extend():** Cho phép xác định kích thước của GridView, tùy vào trường hợp và GridView có thể có 1 hoặc nhiều cột.

**- Ví dụ:**

```
class MyGirdView01 extends StatefulWidget {
  const MyGirdView01({super.key});

  @override
  State<MyGirdView01> createState() => _MyGirdView01State();
}
```

```dart
class _MyGirdView01State extends State<MyGirdView01> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: const Color.fromARGB(255, 234, 205, 18),
        title: const Text('GridView demo 01'),
      ),
      body: GridView.count(
        crossAxisCount: 4,
        crossAxisSpacing: 2,
        mainAxisSpacing: 2,
        children: List.generate(20, (index) {
          return Card(
            child: Container(
              color: Colors.blue,
              child: Center(
                child: Text(
                  'Item $index',
                  style: const TextStyle(fontSize: 20),
                ),
              ),
            ),
          );
        }),
      ),
    );
  }
}
```



## 2. Bài tập hướng dẫn

**Ví dụ 1:** Xây dựng màn hình hiển thị thông tin của học phần lập trình di động với các nội dung như sau :




**Hướng dẫn:** Ta sẽ xây dựng cấu trúc của page với widget tree như sau :

```
class MyListView extends StatefulWidget {
  const MyListView({super.key});

  @override
  State<MyListView> createState() => _MyListViewState();
}

class _MyListViewState extends State<MyListView> {
  void _showDialog(String title, String content) {
    showDialog(
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          title: Text(title),
          content: Text(content),
          actions: [
            TextButton(
              onPressed: () => Navigator.of(context).pop(),
              child: const Text("OK"),
            ),
          ],
        );
      },
    );
  }

  static const TextStyle _textStyle = TextStyle(
    fontSize: 20,
```

```
    color: Colors.red,
    fontWeight: FontWeight.bold,
  );

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("ListView Demo"),
        backgroundColor: const Color.fromARGB(255, 239, 168, 4),
        leading: IconButton(
          icon: const Icon(Icons.home),
          onPressed: () {},
        ),
      ),
      body: ListView(
        children: <Widget>[
          ListTile(
            leading: const Icon(Icons.home),
            title: const Text(
              "Công nghệ phần mềm",
              style: _textStyle,
            ),
            subtitle: const Text("Phát triển các ứng dụng giải quyết các vấn đề thực
tế"),
            trailing: const Icon(Icons.arrow_forward),
            onTap: () {
              _showDialog("Thông báo", "Bạn chọn CNPM");
            },
          ),
          ListTile(
            leading: const Icon(Icons.business),
            title: const Text(
              "Hệ thống thông tin",
              style: _textStyle,
            ),
            subtitle: const Text("Phát triển các kỹ thuật xử lý thông tin trong tổ
chức"),
            trailing: const Icon(Icons.arrow_forward),
            onTap: () {
              _showDialog("Thông báo", "Bạn chọn HTTT");
            },
          ),
          ListTile(
            leading: const Icon(Icons.school),
            title: const Text(
              "Mạng máy tính",
              style: _textStyle,
            ),
            subtitle: const Text("Xử lý các vấn đề liên quan đến mạng máy tính"),
            trailing: const Icon(Icons.arrow_forward),
```

```
            onTap: () {
              _showDialog("Thông báo", "Bạn chọn MMT");
            },
          ),
          ListTile(
            leading: const Icon(Icons.school),
            title: const Text(
              "An toàn thông tin",
              style: _textStyle,
            ),
            subtitle: const Text("Thiết kế và đảm bảo an toàn cho hệ thống máy tính"),
            trailing: const Icon(Icons.arrow_forward),
            onTap: () {
              _showDialog("Thông báo", "Bạn chọn BMTT");
            },
          ),
        ],
      ),
    );
  }
}
```

**Hoặc**

```
class ListItemNew extends StatelessWidget {
  final String title;
  final String content;
  final String dialogContent;
  final Function(String, String) function;

  const ListItemNew({
    super.key,
    required this.title,
    required this.content,
    required this.dialogContent,
    required this.function,
  });

  static const TextStyle _textStyle = TextStyle(
    fontSize: 20,
    color: Colors.red,
    fontWeight: FontWeight.bold,
  );

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        border: Border.all(color: Colors.black),
        color: const Color.fromARGB(255, 200, 197, 177),
        borderRadius: BorderRadius.circular(20),
      ),
```

```
      child: ListTile(
        leading: const Icon(Icons.home),
        title: Text(
          title,
          style: _textStyle,
        ),
        subtitle: Text(content),
        trailing: const Icon(Icons.arrow_forward),
        onTap: () {
          function('Thông báo', 'Bạn chọn $dialogContent');
        },
      ),
    );
  }
}
```

**Và cập nhật lại**

```
class _MyListViewBuilderState extends State<MyListViewBuilder> {
  void _showDialog(String title, String content) {
    showDialog(
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          title: Text(title),
          content: Text(content),
          actions: [
            TextButton(
              onPressed: () => Navigator.of(context).pop(),
              child: const Text("OK"),
            ),
          ],
        );
      },
    );
  }

  static const List dsTieuDe = [
    'Công nghệ phần mềm',
    'Hệ thống thông tin',
    'Mạng máy tính',
    'An toàn thông tin',
  ];

  static const List dsNoiDung = [
    'Phát triển các ứng dụng giải quyết các vấn đề thực tế',
    'Phát triển các kỹ thuật xử lý thông tin trong tổ chức',
    'Xử lý các vấn đề liên quan đến mạng máy tính',
    'Thiết kế và đảm bảo an toàn cho hệ thống máy tính',
  ];

  static const List dsNoiDungTB = ['CNPM', 'HTTT', 'MMT', 'BMTT'];
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("ListView Demo"),
        backgroundColor: const Color.fromARGB(255, 239, 168, 4),
        leading: IconButton(
          icon: const Icon(Icons.home),
          onPressed: () {},
        ),
      ),
      body: ListView.builder(
        itemCount: dsTieuDe.length,
        itemBuilder: (BuildContext context, int index) {
          return ListItemNew(
            title: dsTieuDe[index],
            content: dsNoiDung[index],
            dialogContent: dsNoiDungTB[index],
            function: _showDialog,
          );
        },
      ),
    );
  }
}
```

**Ví dụ 2:** Xây dựng ứng dụng cho phép hiển thị thông tin danh sách các đề tài mà sinh sinh viên có thể lựa chọn theo hình sau :



**Hướng dẫn :**

*file detai.dart*

```
class DeTai {
  final String _maDeTai;
  final String _tenDeTai;
  final String _tenGiangVien;
  final String _noiDung;
  final String _chuyenNganh;

  DeTai({
    required String maDeTai,
    required String tenDeTai,
    required String tenGiangVien,
    required String noiDung,
    required String chuyenNganh,
  })  : _chuyenNganh = chuyenNganh,
        _noiDung = noiDung,
        _tenDeTai = tenDeTai,
        _tenGiangVien = tenGiangVien,
        _maDeTai = maDeTai;

  String get getMaDeTai => _maDeTai;
  String get tenDeTai => _tenDeTai;
  String get tenGiangVien => _tenGiangVien;
  String get noiDung => _noiDung;
  String get chuyenNganh => _chuyenNganh;
}
```

*file detaiItem.dart*

```
class DeTaiItem extends StatelessWidget {
  final DeTai deTai;

  const DeTaiItem({super.key, required this.deTai});

  static const TextStyle _textStyle = TextStyle(
    fontSize: 20,
    color: Colors.red,
    fontWeight: FontWeight.bold,
  );

  @override
  Widget build(BuildContext context) {
    return Container(
      margin: const EdgeInsets.only(left: 10, right: 10, bottom: 5, top: 5),
      height: 230,
      decoration: BoxDecoration(
        border: Border.all(color: Colors.black),
        color: const Color.fromARGB(255, 200, 197, 177),
        borderRadius: BorderRadius.circular(20),
      ),
      child: Column(
```

```
        children: [
          ListTile(
            leading: const Icon(Icons.home),
            title: Text(
              deTai.getMaDeTai,
              style: _textStyle,
            ),
            subtitle: Text(
              deTai.tenDeTai,
              style: const TextStyle(fontSize: 18, color: Colors.blue),
              maxLines: 2,
            ),
            trailing: const Icon(Icons.arrow_forward),
            onTap: () {
              showDialog(
                context: context,
                builder: (BuildContext context) {
                  return AlertDialog(
                    title: const Text('Thông báo'),
                    content: Text('Bạn chọn ${deTai.tenDeTai}'),
                    actions: [
                      TextButton(
                        onPressed: () {
                          Navigator.of(context).pop();
                        },
                        child: const Text('OK'),
                      ),
                    ],
                  );
                },
              );
            },
          ),
          const Divider(
            color: Colors.black,
            height: 5,
            thickness: 2,
            indent: 20,
            endIndent: 20,
          ),
          Padding(
            padding: const EdgeInsets.all(8.0),
            child: Text(
              'Chuyên ngành: ${deTai.chuyenNganh}',
              textAlign: TextAlign.justify,
              maxLines: 3,
              overflow: TextOverflow.ellipsis,
            ),
          ),
          Text(
            'Giáo viên: ${deTai.tenGiangVien}',
```

```
            textAlign: TextAlign.end,
          ),
        ],
      ),
    );
  }
}
```

*file listDeTai.dart*

```
class MyListDeTai extends StatefulWidget {
  const MyListDeTai({super.key});

  @override
  State<MyListDeTai> createState() => _MyListDeTaiState();
}

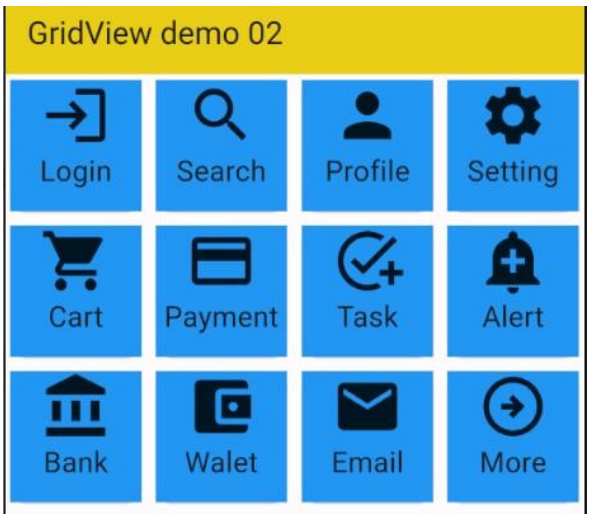
class _MyListDeTaiState extends State<MyListDeTai> {
  void _showDialog(String title, String content) {
    showDialog(
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          title: Text(title),
          content: Text(content),
          actions: [
            TextButton(
              onPressed: () => Navigator.of(context).pop(),
              child: const Text("OK"),
            ),
          ],
        );
      },
    );
  }

  static final List<DeTai> dsDeTai = [
    DeTai(
      maDeTai: "DT01",
      tenDeTai: 'Khai thác tập hữu ích cao trên CSDL giao dịch',
      tenGiangVien: 'ThS. Nguyễn Thanh Truyền',
      noiDung: 'Khai phá tập UNRE, tìm hiểu và vận dụng các kỹ thuật tỉa hiệu quả để
giảm bớt không gian tìm kiếm',
      chuyenNganh: 'CNPM',
    ),
    DeTai(
      maDeTai: "DT03",
      tenDeTai: 'Phát hiện tấn công và ngăn ngừa phát tán mã độc trong hệ thống mạng',
      tenGiangVien: 'ThS. Nguyễn Thanh Truyền',
      noiDung: 'Khai phá topK UNRE, kết hợp với việc tăng ngưỡng sớm để có được minUtil
lớn nhất, từ đó áp dụng các chiến lược tỉa và tăng ngưỡng.',
```

```
      chuyenNganh: 'CNPM',
    ),
    DeTai(
      maDeTai: "DT02",
      tenDeTai: 'Khai thác K tập hữu ích cao nhất (topKUNRE)',
      tenGiangVien: 'TS. Vũ Đức Thịnh',
      noiDung: 'Khai phá topKUNRE, kết hợp với việc tăng ngưỡng sớm để có được minUtil
lớn nhất, từ đó áp dụng các chiến lược tỉa và tăng ngưỡng.',
      chuyenNganh: 'MMT',
    ),
    DeTai(
      maDeTai: "DT04",
      tenDeTai: 'Xây dựng hệ thống thông tin hỗ trợ việc giảng dạy tại UNRET',
      tenGiangVien: 'ThS. Nguyễn Văn Lễ',
      noiDung: 'Khai phá topKUNRE, kết hợp với việc tăng ngưỡng sớm để có được minUtil
lớn nhất, từ đó áp dụng các chiến lược tỉa và tăng ngưỡng.',
      chuyenNganh: 'HTTT',
    ),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("ListView Demo"),
        backgroundColor: const Color.fromARGB(255, 239, 168, 4),
        leading: IconButton(
          icon: const Icon(Icons.home),
          onPressed: () {},
        ),
      ),
      body: ListView.builder(
        itemCount: dsDeTai.length,
        itemBuilder: (BuildContext context, int index) {
          return DeTaiItem(
            deTai: dsDeTai[index],
          );
        },
      ),
    );
  }
}
```

**Ví dụ 3:** Xây dựng ứng dụng cho phép hiển thị thông tin danh sách các đề tài mà sinh sinh viên có thể lựa chọn theo hình sau :




**Hướng dẫn :**

Xây dựng lớp GirdItem

```
class GirdItem {
  final String title;
  final IconData icon;

  GirdItem({required this.title, required this.icon});
}
```

Hiển thị GridView sử dụng GridView.builder()

```
class MyGirdView02 extends StatefulWidget {
  const MyGirdView02({super.key});

  @override
  State<MyGirdView02> createState() => _MyGirdView02State();
}

class _MyGirdView02State extends State<MyGirdView02> {
  final List<GirdItem> lst = [
    GirdItem(title: 'Login', icon: Icons.login),
    GirdItem(title: 'Search', icon: Icons.search),
    GirdItem(title: 'Profile', icon: Icons.person),
    GirdItem(title: 'Setting', icon: Icons.settings),
    GirdItem(title: 'Cart', icon: Icons.shopping_cart),
    GirdItem(title: 'Payment', icon: Icons.payment),
    GirdItem(title: 'Task', icon: Icons.add_task),
    GirdItem(title: 'Alert', icon: Icons.add_alert),
    GirdItem(title: 'Bank', icon: Icons.account_balance),
    GirdItem(title: 'Walet', icon: Icons.account_balance_wallet),
    GirdItem(title: 'Email', icon: Icons.mail),
    GirdItem(title: 'More', icon: Icons.arrow_circle_right_outlined),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

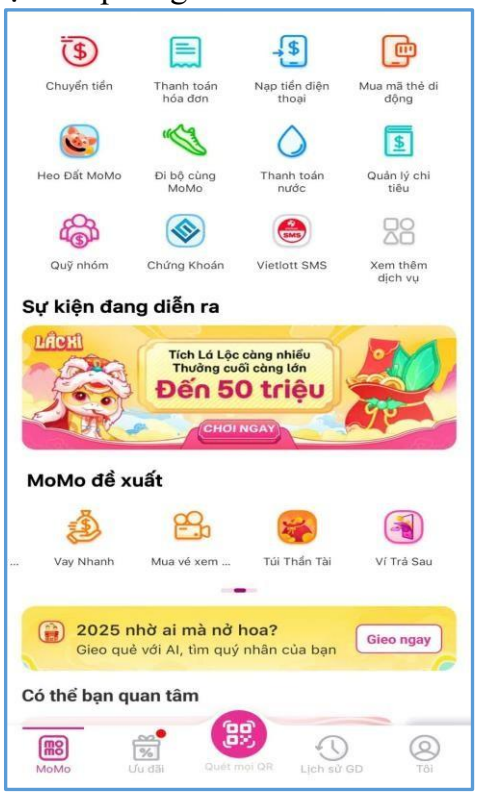
```
      appBar: AppBar(
        backgroundColor: const Color.fromARGB(255, 234, 205, 18),
        title: const Text('GridView demo 02'),
      ),
      body: GridView.builder(
        itemCount: lst.length,
        gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
          crossAxisCount: 4,
          crossAxisSpacing: 2,
          mainAxisSpacing: 2,
        ),
        itemBuilder: (context, index) {
          return Card(
            child: Container(
              color: Colors.blue,
              child: Center(
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    Icon(lst[index].icon, size: 50),
                    Text(lst[index].title, style: const TextStyle(fontSize: 20)),
                  ],
                ),
              ),
            ),
          );
        },
      ),
    );
  }
}
```

### 3. Bài tập tại lớp :

**Bài tập 01.** Xây dựng giao diện cho phép người dùng lựa chọn theo giao diện sau :

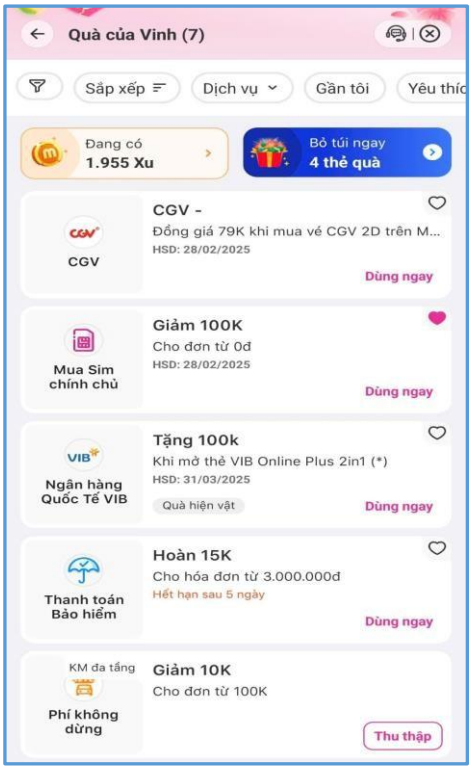



**Bài tập 02.** Xây dựng giao diện mô phỏng màn hình của Momo như sau :




.

**Bài tập 03.** Xây dựng giao diện mô phỏng màn hình sau :

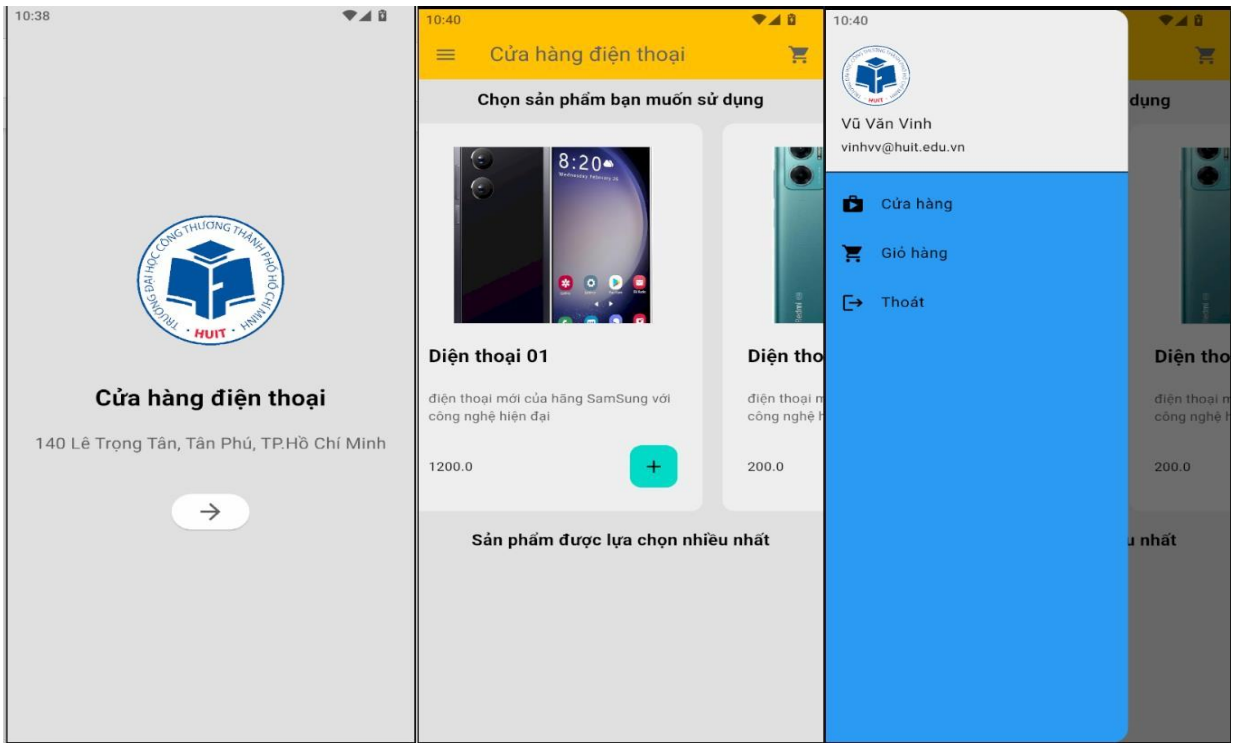



## 4. Bài tập về nhà :

**Bài tập 04. Tham khảo** Xây dựng ứng dụng mua bán điện thoại đơn giản với các màn hình như sau :

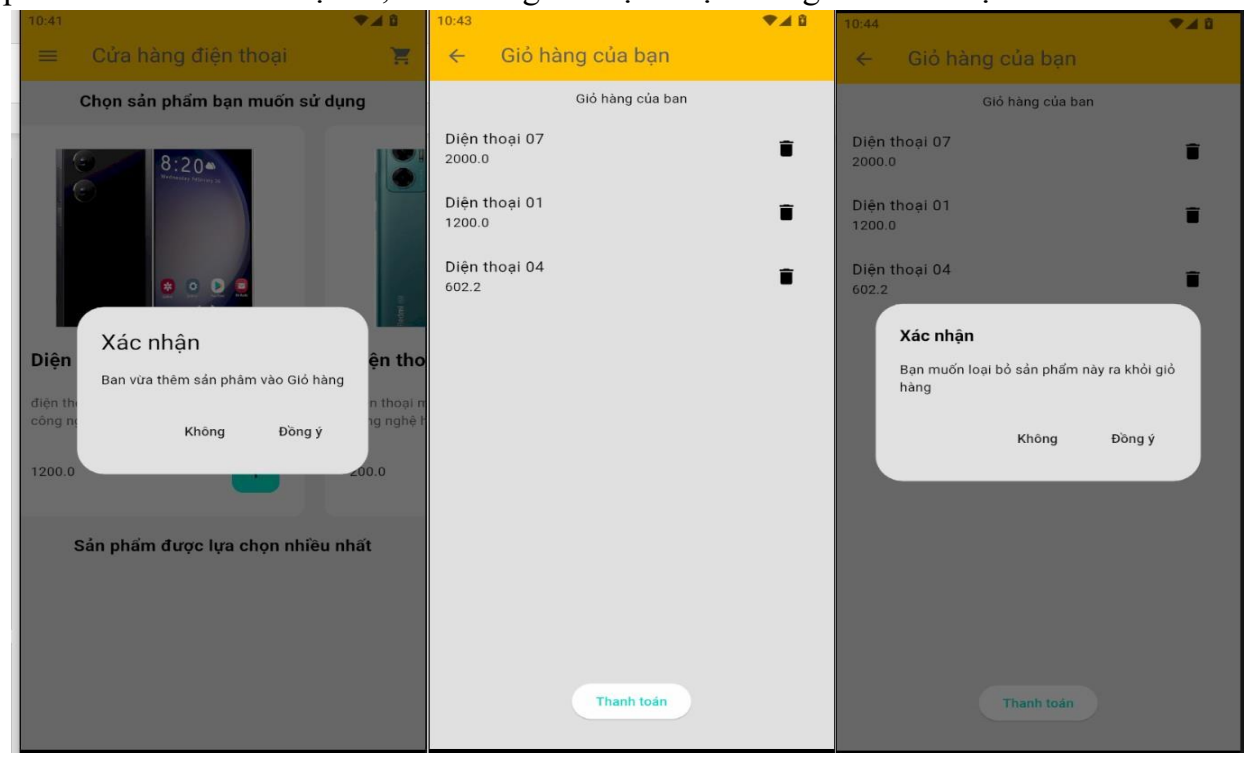
Mô tả : Khi chương trình được thực thi, màn hình giới thiệu ứng dụng được hiện lên như hình. Người dùng bấm vào button mũi tên để vào ứng dụng, màn hình hiển thị sản phẩm. Màn hình ứng dụng hiển thị sản phẩm sẽ hiển thị điện thoại mà cửa hàng đang bán với các thông tin như màn hình. Trong màn hình sẽ có thiết kế Drawer như trong hình minh họa

- Giỏ hàng : hiển thị thông tin giỏ hàng
- Cửa hàng : là trang hiển thị sản phẩm
- Thoát : là trang giới thiệu




Khi khác hàng muôn mua sản phẩm nào thì bấm vào button. Khi khách hàng chọn mua một sản phẩm thì sẽ có hội thoại xác nhận. Khách hàng có thể chọn nhiều sản phẩm. Khi khách hàng hoàn thành việc mua hàng, khác hàng có thể bấm vào biểu tượng giỏ hàng trên thanh công cụ hoặc bấm vào Giỏ hàng trong Drawer.

Màn hình giỏ hàng sẽ hiển thị thông tin của các sản phẩm mà khách hàng đã chọn. Nếu khách hàng chưa chọn sản phẩm nào thì sẽ thông báo là « Bạn chưa có sản phẩm nào trong giỏ hàng ». Với mỗi sản phẩm trong giỏ hàng, khách hàng có thể bấm vào biểu tượng để loại bỏ sản phẩm đó. Trước khi loại bỏ, khác hàng sẽ nhận được thông báo xác nhận như hình bên dưới



Trong màn hình giỏ hàng, người dùng có thể bấm vào button thanh toán để xác nhận thanh toán hoặc trở lại trang sản phẩm để mua thêm sản phẩm.

